Name:Gurshan Singh Brar Roll no:1024170270 Batch:2Q12

1)
```
#include <iostream>
using namespace std;

int main() {
   int start[]  = {1, 3, 0, 5, 8, 5};
   int finish[] = {2, 4, 6, 7, 9, 9};
   int n = 6;

   int count = 1;
   int lastFinish = finish[0];
   for (int i = 1; i < n; i++) {
      if (start[i] >= lastFinish) {
         cout << i << " ";
         count++;
         lastFinish = finish[i];
      }
   }
   cout <<  count;

   return 0;
}
```

```
1 3 4 4
```

2)
```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
   int arr[] = {900, 940, 950, 1100, 1500, 1800};
   int dep[] = {910, 1200, 1120, 1130, 1900, 2000};
   int n = 6;
   sort(arr, arr + n);
```

```
    sort(dep, dep + n);

    int platforms = 1;
    int maxPlatforms = 1;
    int i = 1, j = 0;
    while (i < n && j < n) {
        if (arr[i] <= dep[j]) {
            platforms++;
            i++;
        } else {
            platforms--;
            j++;
        }
        if (platforms > maxPlatforms)
            maxPlatforms = platforms;
    }
    cout << "Minimum number of platforms " << maxPlatforms;
    return 0;
}
```

```
Minimum number of platforms 3
```

3)
```
#include <iostream>
#include <algorithm>
using namespace std;
class Job {
public:
    char id;
    int deadline;
    int profit;
};

bool compare(Job a, Job b) {
```

```
    return a.profit > b.profit;
}
int main() {
    Job jobs[] = {
        {'a', 4, 20},
        {'b', 1, 10},
        {'c', 1, 40},
        {'d', 1, 30}
    };
    int n = 4;
    sort(jobs, jobs + n, compare);

    bool slot[5] = {false};
    int totalProfit = 0;

    cout << "Selected jobs ";

    for (int i = 0; i < n; i++) {
        for (int j = jobs[i].deadline; j > 0; j--) {
            if (!slot[j]) {
                slot[j] = true;
                totalProfit += jobs[i].profit;
                cout << jobs[i].id << " ";
                break;
            }
        }
    }
    cout << endl;
    cout << "Maximum profit " << totalProfit;
    return 0;
```

```
Selected jobs c a
Maximum profit 60
```

4)
```
#include <iostream>
#include <algorithm>
using namespace std;

class Item {
public:
    int profit;
    int weight;
};
bool compare(Item a, Item b) {
    return (a.profit / a.weight) > (b.profit / b.weight);
}
int main() {
    Item arr[] = {{60, 10}, {100, 20}, {120, 30}};
    int n = 3;
    int W = 50;

    sort(arr, arr + n, compare);

    int totalProfit = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i].weight <= W) {
            W = W - arr[i].weight;
            totalProfit = totalProfit + arr[i].profit;
        } else {
            totalProfit = totalProfit + (arr[i].profit * W) / arr[i].weight;
            break;
```

```
        }
    }
    cout << "Maximum profit " << totalProfit;
    return 0;
}
```

```
Maximum profit 240
```

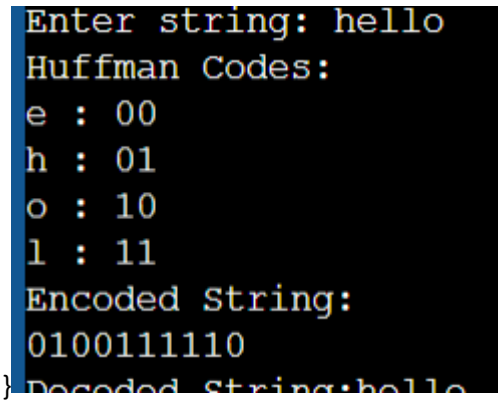
5)

```
#include <iostream>
#include <queue>
using namespace std;
class Node {
public:
    char ch;
    int freq;
    Node *left, *right;

    Node(char c, int f) {
        ch = c;
        freq = f;
        left = right = NULL;
    }
};
class Compare {
public:
    bool operator()(Node* a, Node* b) {
        return a->freq > b->freq;
    }
};
char characters[100];
string codes[100];
int codeCount = 0;
```

```
void storeCode(char c, string code) {
    characters[codeCount] = c;
    codes[codeCount] = code;
    codeCount++;
}
string getCode(char c) {
    for (int i = 0; i < codeCount; i++) {
        if (characters[i] == c)
            return codes[i];
    }
    return "";
}
void generateCodes(Node* root, string code) {
    if (!root)
        return;

    if (!root->left && !root->right)
        storeCode(root->ch, code);

    generateCodes(root->left, code + "0");
    generateCodes(root->right, code + "1");
}
string encode(string text) {
    string result = "";
    for (char c : text)
        result += getCode(c);
    return result;
}
string decode(string encoded, Node* root) {
    string result = "";
```

```
    Node* curr = root;

    for (char bit : encoded) {
        if (bit == '0')
            curr = curr->left;
        else
            curr = curr->right;

        if (!curr->left && !curr->right) {
            result += curr->ch;
            curr = root;
        }
    }
    return result;
}
int main() {
    string text;
    cout << "Enter string: ";
    getline(cin, text);

    int freq[256] = {0};
    for (char c : text)
        freq[(int)c]++;
    priority_queue<Node*, vector<Node*>, Compare> pq;

    for (int i = 0; i < 256; i++) {
        if (freq[i] > 0)
            pq.push(new Node((char)i, freq[i]));
    }
    while (pq.size() > 1) {
        Node* left = pq.top(); pq.pop();
```

```
    Node* right = pq.top(); pq.pop();

    Node* parent = new Node('$', left->freq + right->freq);
    parent->left = left;
    parent->right = right;

    pq.push(parent);
  }
  Node* root = pq.top();
  generateCodes(root, "");
  cout << "Huffman Codes:\n";
  for (int i = 0; i < codeCount; i++)
    cout << characters[i] << " : " << codes[i] << endl;
  string encoded = encode(text);
  cout << "Encoded String:\n" << encoded << endl;
  string decoded = decode(encoded, root);
  cout << "Decoded String:" << decoded << endl;
  return 0;
```



}